

Bimanual reachability of three UR7e layouts

Question: which layout can actually *work* the shared volume with both arms at once? **Test:** every one of 450 waypoints, one verdict per waypoint per layout — both arms reach it, touch nothing on the way in, and hold it parked.

Produced by `bimanual_test.py` from `workspace_benchmark.json`; per-waypoint verdicts in `bimanual_results.json`. Companion to `TORQUE_REPORT.md`, which finds the three layouts equivalent on actuator load — so this is the axis that should decide the layout.

Supersedes an earlier version run on a 45-waypoint grid. The grid is now 10× denser (450 points, uniform 5 cm spacing), and that changed one conclusion: `urgantry` no longer has zero collisions. See *What the denser grid changed*.

Headline

	<code>urgantry</code>	<code>urtable_45</code>	<code>urtable</code>
PASS, of 450	379	378	344
<code>collision</code>	9	37	60
<code>droop</code> (parked >20 mm off)	62	33	46
<code>unstable</code> / no IK	0 / 0	1 / 1	0 / 0

`urgantry` and `urtable_45` are tied — one waypoint apart in 450 is not a difference. `urtable` trails by ~35 waypoints, and every one of its collisions is the two arms hitting *each other*.

Three things matter more than the PASS column:

1. **Kinematic reach is not the constraint.** One `no IK` across 1350 waypoint-layout pairs. Nothing here is out of the arms' range; failures are interference, sag, or posture.
2. **The failure modes are qualitatively different**, and that should drive the decision more than the totals — see below.
3. **283 of 450 waypoints pass in all three layouts**, and 28 pass in none. Only `urtable_45` passes anything the others cannot (26 waypoints); `urgantry` and `urtable` each uniquely pass **zero**.

Failure modes

`urtable` — the arms collide with each other

All **60** of its collisions are arm-vs-arm; not one involves the table or its own structure:

pair	count
left_wrist_1_link ↔ right_wrist_1_link	20
left_forearm_link ↔ right_wrist_1_link	20
left_wrist_1_link ↔ right_forearm_link	20

Perfectly mirrored, which is what a symmetric layout should produce. They cluster with **height**: 5 collisions in the low layer, 17 in mid, **38 in high**. Reaching up forces both arms into the same volume above the 20 cm gap between their bases.

This is the expensive failure to fix. It is not a clearance detail — it is the base spacing, so the remedy is moving the bases apart, which changes the footprint.

urtable_45 — the arms hit their own stand

35 of 37 collisions are arm-vs-structure: gripper couplers into `mount_pedestal`, forearms and wrists into the `board`. Only 2 are arm-vs-arm. They sit in the **front rows** (mean $y = -0.21$ against a grid mean of 0.00) — poses that reach back over the angled stand the arms are mounted on.

This is the cheap failure to fix: it is the fixture, not the kinematics. Reshaping or lowering the pedestal recovers most of those waypoints without touching arm placement.

urgantry — clean, but it sags

Only **9** collisions (7 structural — upper arm into its own column and head, gripper into the board edge; 2 arm-vs-arm). Its 62 failures are almost all `droop`: the arm arrives and settles, but parks more than 20 mm from the goal.

Droop clusters in the **far row** (mean $y = +0.19$, grid mean 0.00) and toward the outside in x (mean $|x| = 0.25$ vs 0.19) — the poses with the longest base-to-goal distance, where the position servo has least authority against gravity.

Droop is a soft failure, and the gate is close

	urgantry	urtable	urtable_45
median droop	22 mm	22 mm	22 mm
p90	26 mm	25 mm	52 mm
max	29 mm	28 mm	85 mm
would pass at a 30 mm gate	62 of 62	46 of 46	28 of 33

Almost every droop failure is a near-miss of the 20 mm tolerance. **At a 30 mm gate the standings become** `urgantry` **441**, `urtable_45` **406**, `urtable` **390 of 450**. Whether droop counts as failure depends entirely on the tolerance the task needs — it is a servo stiffness property, not a workspace one. `urtable_45` is the only layout with genuinely bad droop cases (up to 85 mm).

Per-layer breakdown

PASS out of the 150 waypoints in each height layer (`low` = +0.05 m above the board, `mid` = +0.20, `high` = +0.35):

	low	mid	high
<code>urgantry</code>	117	130	132
<code>urtable_45</code>	110	124	144
<code>urtable</code>	123	123	98

Height splits the layouts in opposite directions. `urtable_45` is the best layout in the air (144/150 high) and the worst near the board (110/150) — the angled stand lifts the arms above the work surface, which costs low reaches and buys high ones. `urtable` is the opposite and collapses at height (98/150), for the arm-vs-arm reason above. `urgantry` is the most uniform across heights, which is worth something on its own if the task spans the volume.

What the denser grid changed

The 45-waypoint version reported `urgantry` with **zero collisions**. At 450 points it has 9 — including its upper arm striking its own column. That was a sampling artifact: the coarse grid never placed a waypoint where the interference occurs. Nothing else moved much; PASS rates per layout are within a few percent of the sparse run.

The lesson generalises: a collision-free result on a coarse grid is weak evidence. The denser grid also sharpened `urtable`'s diagnosis from "9 collisions" to "60, all arm-vs-arm, concentrated at height", which is a much clearer design signal.

Recommendation

`urgantry` and `urtable_45` are tied on the numbers; pick on failure mode.

- Choose `urtable_45` if the work is in the air. It wins the high layer outright (144/150), it is the only layout that uniquely serves any waypoints, and its main failure is a fixture shape that can be redesigned.
- Choose `urgantry` if the work spans the full height range. It is the most uniform layer to layer, has the fewest collisions of any kind, and all of its failures are sub-30 mm droop that a stiffer controller or a looser tolerance erases.
- `urtable` is the weakest of the three and its problem is structural: 20 cm base spacing puts the arms in each other's way exactly where the task is most demanding.

Method

- Both arms serve every waypoint **simultaneously**, each offset from it by 0.35 m along the base-to-base axis. No single-arm scoring.

- PASS requires all of: both arms solve IK to their goal tool-down; **zero contacts involving either arm at any point of the approach**, not merely at the end; both settle ($\max |\dot{q}| < 0.01 \text{ rad/s}$) within 20 mm; and stay parked and contact-free for a further second.
- Verdicts: `no IK`, `collision` (recording the geom pair), `unstable`, `droop`, `drifted`, `PASS`.
- Loose props (block, tray) are removed from the scene, so the only things an arm can hit are the fixed structure, the table, and the other arm.
- **implicitfast integrator.** With the scenes' default Euler, the stiff position servos (kp 2000 / kv 400) chatter at the timestep frequency — joints flip $\pm 0.0008 \text{ rad}$ every step with $\sim 7 \mu\text{rad}$ of net drift, reading as 0.39 rad/s and failing any settle test.
- Grid: 450 waypoints on a uniform 5 cm lattice — 15 in x ($\pm 0.35 \text{ m}$) $\times$ 10 in y ($\pm 0.225 \text{ m}$) $\times$ 3 height layers, in the table frame.

Caveats

- **A `collision` verdict is evidence, not proof.** The IK is damped least squares warm-started from the current pose, with random restarts if that fails, accepting the *first* restart that converges. It does not enumerate the ~ 8 analytic UR branches or prefer the least-motion one — on `urtable_45`, ~ 22 – 26 distinct collision-free solutions exist per pose, and a few percent of solves take a branch needing $> 1 \text{ rad}$ more travel than the best available. There is also no path planning: joints interpolate independently, so the approach path is whatever that produces. A different solver or a planned trajectory could clear some of these collisions.
- **`droop` is a controller property, not a workspace one.** It depends on servo gains and on the 20 mm tolerance, both of which are choices.
- **Static poses only.** No payload, no motion through the waypoints, no task forces.
- Simulation: no friction model, no gearbox compliance, no joint backlash.